C291 - Spring 26 - System Programming with C & Unix
Assignment 8

Q.1 String Tokenizer (20 marks)

Write a program in C with a function **char ** myTokenizer(char * str, const char *delim)** that breaks a string into individual tokens(words) based upon the provided delimiter and returns these tokens. **It is pivotal that you do not use the strtok function in this package. Doing so will result in 0 marks given.**

The argument to the function are **char * str**, the input string to tokenize, and **const char * delim**, the set of characters to be used as individual delimiters.
The function will return a **char** **, an array of char pointers to char arrays. **If no tokens were found, return NULL.**

It must be noted that the characters in the delimiter are consumed, i.e., they are not included in the returned tokens.

The input string **str** and the delimiter **delim** will be provided by the user. Your program should prompt the users for these inputs and print the tokens, one per line. Because scanf will split your inputs by whitespace, it is acceptable to use fgets for this input.

Your program must be case insensitive.

Example 1:
Input:
Input the string: This is the C291 homework!
Input the delimiter: 291
Output:
This is the C
homework!

Example 2:

Input:

Input the string: The Indiana University campus rests on a bed of Indiana limestone.

Input the delimiter: ON

Output:

The I

dia

a U

iversity campus rests

a bed

f I

dia

a limest

e.

Example 3:

Input:

Input the string: The quick brown fox jumps over the lazy dog.

Input the delimiter: and

Output:

The quick brow

fox jumps over the l

zy

og.

Note: The solution for this problem should be named as solution_1.c and it should be under C291-Spring-25/assignment8 where assignment8 would be the new directory you would create.

Q.2 Index Of (10 Marks)

Write a program in C with a function **int indexOf(char * haystack, char * needle, int fromIndex)** that returns the index of the first occurrence of **needle** in the **haystack** starting from **fromIndex**. The program should prompt user to input the haystack and needle strings and print the output.

Example 1:

Input:

Haystack: sad but true

Needle: sad

From Index: 0

Output:

The needle is found at index 0.

Example 2:

Input:

Haystack: it is a wonderful day

Needle: wonder

From Index: 2

Output:

The needle is found at index 8.

Example 3:

Haystack: it is a wonderful day

Needle: xyz Output:

From Index: 0

Output:

The needle cannot be found in the haystack.

Note: The solution for this problem should be named as solution_2.c and it should be under C291-Spring-25/assignment8 where assignment8 would be the new directory you would create.

Q.3 Replace All (20 Marks)

Write a C program with a function **char * replaceAll(const char * string, char * target, char * replacement)** that replaces all occurrence of **target** with **replacement** in **string** and returns a pointer to the resulting string. Your program should prompt the user to provide the three input strings.

Your program must be case sensitive.

Example 1:
Input:
Enter a string: It's been a wonderful day.
Enter the target: been
Enter replacement: was

Output: It's was a wonderful day.

Example 2:
Input:
Enter a string: This is the C291 homework!
Enter the target: homework
Enter replacement: assignment

Output: This is the C291 assignment!

Example 3:
Input:
Enter a string: The quick brown fox jumps over the lazy dog.
Enter the target: That
Enter the Replacement: homework
Output: The quick brown fox jumps over the lazy dog.
Explanation: Since the target cannot be found in the original string we return the original string itself.

Note: The solution for this problem should be named as solution_3.c and it should be under C291-Spring-25/assignment8 where assignment8 would be the new directory you would create.

Q.4 XML parser (20 Marks)

Write an XML parser that given an XML string consisting of a book catalog, parse the XML and prints the information of the most and the least expensive books. Use the starter code provided with the assignment for this problem. The starter code includes the XML provided as a string literal

The following is an excerpt from the XML.

```
<catalog>
  <book id=\"bk101\">
    <author>Gambardella, Matthew</author>
    <title>XML Developer's Guide</title>
    <genre>Computer</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description>An in-depth look at creating applications with XML.
</description>
  </book>
  <book id=\"bk102\">
    <author>Cormen, Stein</author>
    <title>Introduction to Algorithm, fourth edition</title>
    <genre>Computer</genre>
    <price>84.89</price>
    <publish_date>2022-04-05</publish_date>
    <description>A comprehensive update of the leading algorithms text, with new material on matchings in bipartite graphs, online algorithms, machine learning, and other topics.
    </description>
  </book>
  <book id=\"bk103\">
    <author>Carl Ryan</author>
    <title>Python Programming for beginners: A Crash Course</title>
    <genre>Computer</genre>
    <price>7.11</price>
    <publish_date>2022-01-19</publish_date>
    <description>The ultimate guide for beginners</description>
  </book>
</catalog>
```

Output:

Most Expensive Book:
Author: Cormen, Stein
Title: Introduction to Algorithm, fourth edition
Genre: Computer
Price: 84.49
Publication Date: 2022-04-05
Description: A comprehensive update of the leading algorithms text, with new material on matching in bipartite graphs, online algorithms, machine learning, and other topics.

Least Expensive Book:
Author: Carl Ryan
Title: Python Programming for Beginners: A Crash Course
Genre: Computer
Price: 7.11
Publication Date: 2022-01-19
Description: The ultimate guide for beginners

Note: The solution for this problem should be named as solution_4.c and it should be under C291-Spring-25/assignment8 where assignment8 would be the new directory you would create.

Q.5 Search Engine (30 marks)

Implement a basic search engine that given a set of documents and a query, finds the best matching document. In your program, write a function **char * searchEngine(char ** documents, char * query, int S)** that returns the first **S** characters from the document in **documents** that best match the **query**. The algorithm is provided below.

The documents are provided at the end of this problem description and should be hard coded in your program (like the questions in assignment 6). Your program should prompt the user to provide the query that may contain multiple words. The search engine should be case insensitive.

Search Engine Algorithm

// A simple search engine
// For a given mult i word query, returns the first S characters from the top-ranking document

Input:
char * document[N] // N is the number of documents
char * query // q is the number of terms in the query
int S // the number of characters (from the beginning) from the resulting document to return

Output:
char * result;

Variables:
int scores[q][N] // initialize all values to 0
int intersection[N] // initialize all values to 0

// Compute the score of each query term in each document

For each term in the query
For each document in documents
Compute the frequency f_{ij} of the term in the document
scores[i][j] = f_{ij}

// Compute the intersection of the rows of scores

```
For each column j in scores
  If all the rows in this column have nonzero values
   store their sum in intersection[j]

// Find the top-ranking document

Find the maximum value in intersection and return its index m
Store the first S characters from document[m] in result
Return result
```

Documents

[0] Located in Bloomington, Indiana, the campus of IU Bloomington is home to an extensive student life program. The university campus is 1937 acres, the largest of any university in Indiana. The university has a student count of over 45000. The campus is where a student takes classes, just like any other campus.

[1] The campus rests on a bed of Indiana limestone, along with the rest of Bloomington and southern Indiana. The Bryan house is the campus home of the university president. To a student and a faculty member alike, Indiana University has a campus that is rich with green space and historic buildings. Bloomington, Indiana is located in Southern Indiana.

[2] The Indiana Memorial Union in Bloomington is the second largest student union in the world. It features a student activities tower, which houses a variety of student organizations. The University campus is centered around this student union. The Campus has an extensive bus system.

[3] The Luddy School of Informatics, Computing, and Engineering is a school within Indiana University. A student can major in fields such as Informatics and Computer Science. The student count has grown greatly in the past few years, and Luddy Hall was built on Indiana University campus in 2018. Indiana University does not have a mascot. The university is known for its Sample Gates.

Example 1:

Query: Indiana campus
Input number of characters to return: 50

Output:
The campus rests on a bed of Indiana limestone, al

Explanation:
The **scores** array will be as follows
Rows represent the query terms, Indiana and campus. Columns represent the Documents. The numbers at each cell represent the frequency of the terms Indiana and campus in each document, e.g. Indiana was found 2 times in Documents[0], 5 times in Document[1], one time in Document[2] and 3 times in Document[3]

2	5	1	3
4	3	2	1

The corresponding **intersection** array will be:

6	8	3	4

Each number in the cell correspond to the sum of nonzero values in each column.

The maximum value in the intersection array is 8 which is at index 1, therefore Documents[1] will be returned.

Example 2:

Query: Bloomington
Input number of characters to return: 50

Output:
Located in Bloomington, Indiana, the campus of IU

Explanation:

The **scores** array will be as follows

There is only one row representing the query term, bloomington. Columns represent the Documents. The numbers at each cell represent the frequency of the term bloomington in each document, e.g. bloomington was found 2 times in Documents[0], 2 time in Document[1], one time in Document[2] and 0 times in Document[3]

2	2	1	0

The corresponding **intersection** array will be:

2	2	1	0

Since there is only one row in scores, the intersection will be the same as scores, as there are no other row to add.

The maximum value in the intersection array is 2 which is at index 0 and 1. We will return the document at the smaller index, i.e. Documents[0]

Example 3:

Query: bloomington indiana
Input number of characters to return: 50

Output:

The campus rests on a bed of Indiana limestone, al

Explanation:

The **scores** array will be as follows

Rows represent the query terms, bloomington and indiana. Columns represent the Documents. The numbers at each cell represent the frequency of the terms bloomington and indiana in each document

2	2	1	0
2	5	1	3

The corresponding **intersection** array will be:

4	7	2	0

Each number in the cell correspond to the sum of nonzero values in each column. Since scores[0][3] is 0, the sum of column 3 will not be computed and it will remain 0 (initial value)

The maximum value in the intersection array is 7 which is at index 1, therefore Documents[1] will be returned.

Note: The solution for this problem should be named as solution_5.c and it should be under C291-Spring-25/assignment8 where assignment8 would be the new directory you would create.